

Módulo 2 · Prueba Técnica Práctica

Solución implementada (Spring Boot + React) y formato de entrega: enlace público de GitHub.

1. Cómo funciona la solución

El ejercicio se resolvió con **React 19 + Vite + TypeScript** (frontend) y dos backends con **contrato idéntico**: una **API en memoria Express/tsx** (`server.ts` , puerto 3000) que replica cada endpoint y regla para que la demo funcione 100 % sin infraestructura, y el **backend real Spring Boot 3 + Java 17** (`spring-boot-backend/` , puerto 8080) con capas `controller` → `service` → `repository` sobre H2. Ambos exigen el header `x-api-key: 123456` (401 si falta) y notifican cada cambio al CORE vía `POST /core-mock/evento` .

2. Los 6 endpoints obligatorios (backend Spring Boot)

Endpoint	Descripción	Implementación
GET /polizas (tipo, estado)	Listar pólizas por filtros	PolizaRepository.findByTipoAndEstado (JPQL con predicados opcionales)
GET /polizas/{id}/riesgos	Riesgos de la póliza	RiesgoService.obtenerRiesgosPorPoliza (404 si la póliza no existe)
POST /polizas/{id}/renovar	Renovar con ajuste IPC; estado → RENOVADA	PolizaServiceImpl.renovarPoliza: $\text{canon} \times (1 + \text{IPC}/100)$, $\text{prima} = \text{canon} \times \text{vigencia}$; evento RENOVACION_IPC
POST /polizas/{id}/cancelar	Cancelar póliza en cascada	PolizaServiceImpl.cancelarPoliza: cancela la póliza y todos sus riesgos; evento CANCELACION_POLIZA_Y_RIESGOS
POST /polizas/{id}/riesgos	Agregar riesgo (solo colectiva; 1er riesgo si individual)	RiesgoServiceImpl.agregarRiesgoAPoliza con validación de tipo y conteo; recalcula canon acumulado en colectiva
POST /riesgos/{id}/cancelar	Cancelar riesgo individual	RiesgoServiceImpl.cancelarRiesgo; resta el canon en colectiva; evento CANCELAR_RIESGO

Extras implementados: `GET /polizas/{id}` , `POST /polizas` (emisión con cálculo de prima y riesgo inicial) y el mock `POST /core-mock/evento` .

3. Reglas de negocio y códigos de respuesta

Regla	Implementación	HTTP
Individual solo puede tener 1 riesgo activo	<code>countByPolizaIdAndEstado ≥ 1</code> → <code>BusinessException</code>	422
No se puede renovar una póliza CANCELADA	<code>PolizaServiceImpl.renovarPoliza</code>	422
No se pueden agregar riesgos a póliza CANCELADA	<code>RiesgoServiceImpl.agregarRiesgoAPoliza</code>	422
Cancelar póliza cancela todos sus riesgos	Cascada: <code>findByPolizaId</code> + cancelar cada riesgo	200
Agregar riesgo valida el tipo de póliza	INDIVIDUAL con 1 riesgo activo → rechazado	422
Falta/erróneo x-api-key	<code>ApiKeyAuthFilter</code> (filtro HTTP @Order(1))	401

Regla	Implementación	HTTP
Recurso inexistente	ResourceNotFoundException	404
Validación de campos (Bean Validation)	MethodArgumentNotValidException	400
Error no controlado	Excepción genérica	500

Los códigos los centraliza `GlobalExceptionHandler` (`@RestControllerAdvice`), y toda respuesta usa el envoltorio `ApiResponse{exito, mensaje, datos, timestamp}`.

4. Autenticación y cookies — seguridad real

El `ApiKeyAuthFilter` valida el header en cada petición (permitiendo solo `/h2-console` y `/favicon.ico`). La clave se configura externamente en `application.properties` (`app.security.api-key`) y se inyecta con `@Value`.

5. Ejemplos de consumo (cURL)

Puesto que enviamos el proyecto completo (frontend + backend) a su repositorio público, los endpoints se consumen así:

```
# 1. Listar pólizas filtradas
curl -X GET "http://localhost:8080/polizas?tipo=COLECTIVA&estado=ACTIVA" -H "x-api-key: 123456"

# 2. Riesgos de una póliza
curl -X GET "http://localhost:8080/polizas/1/riesgos" -H "x-api-key: 123456"

# 3. Renovar con IPC 9,28 %
curl -X POST "http://localhost:8080/polizas/1/renovar" -H "x-api-key: 123456" \
-H "Content-Type: application/json" -d '{"porcentajeIpc": 9.28}'

# 4. Cancelar póliza (cancela sus riesgos)
curl -X POST "http://localhost:8080/polizas/1/cancelar" -H "x-api-key: 123456"

# 5. Agregar riesgo a póliza colectiva
curl -X POST "http://localhost:8080/polizas/2/riesgos" -H "x-api-key: 123456" \
-H "Content-Type: application/json" -d '{
  "direccion": "Carrera 7 # 116-50 Apto 802",
  "ciudad": "Bogota D.C.",
  "valorCanon": 3200000.00,
  "descripcionInmueble": "Apartamento estrato 5 con balcon",
  "arrendatarioNombre": "Camila Montoya",
  "arrendatarioDoc": "1020304050"
}'

# 6. Cancelar riesgo individual
curl -X POST "http://localhost:8080/riesgos/3/cancelar" -H "x-api-key: 123456"

# Mock de sincronización WebLogic CORE
curl -X POST "http://localhost:8080/core-mock/evento" -H "x-api-key: 123456" \
-H "Content-Type: application/json" -d '{"evento": "ACTUALIZACION", "polizaId": 555}'
```

6. Estructura, instalación y ejecución

6.1 Frontend + API en memoria (app completa, :3000)

```
npm install
cp .env.example .env.local      # opcional (GEMINI_API_KEY solo para funciones IA)
npm run dev                     # abrir http://localhost:3000
```

6.2 Backend Spring Boot real (:8080)

```
cd spring-boot-backend
mvn clean package
mvn spring-boot:run
# Consola H2: http://localhost:8080/h2-console (jdbc:h2:mem:polizasdb · sa · sin password)
```

7. Formato de entrega: enlace público de GitHub

Enlace del repositorio público (completar antes de enviar)

<https://github.com/tu-usuario/tu-repositorio>

El formato solicitado es un **enlace público de GitHub** — no se envían archivos, carpetas ni enlaces a Drive con el código fuente.

Guía para publicar la solución en GitHub

```
# 1. Desde la raíz del proyecto (frontend + spring-boot-backend juntos)
git init
git add .
git commit -m "feat: plataforma gestion de polizas de arrendamiento (modulos 1-4)"

# 2. Crear el repositorio público y subirlo (con GitHub CLI) - alternativa A
gh auth login
gh repo create seguros-polizas --public --push --source . --remote origin

# 2b. Alternativa B: usar un repositorio ya creado en github.com
git remote add origin https://github.com/tu-usuario/tu-repositorio.git
git branch -M main
git push -u origin main
```

- **.gitignore:** excluir `node_modules/`, `dist/`, `target/`, `.env.local` y cualquier archivo con claves reales.
- **No subir credenciales:** la clave `x-api-key` de la demo está documentada (123456) a propósito; no incluyas claves de Gemini ni datos sensibles.
- **Verificación final:** que el README y ambos proyectos queden visibles y compilables desde el repositorio público.

Evidencia de código: `spring-boot-backend/src/main/java/com/seguros/polizas` (controller, service, repository, security, exception, dto, model) y `server.ts` (demo Express en memoria).